Documentación del Firmware de Volante SimRacing (Raspberry Pi Pico W)

Este documento registra el proceso completo de desarrollo, optimización y prueba del firmware para un volante de carreras tipo **Fórmula / GT** basado en la placa **Raspberry Pi Pico W**, programado mediante el **Arduino IDE** utilizando la pila USB nativa **Adafruit TinyUSB**.

El objetivo de este proyecto es obtener un dispositivo **USB HID Gamepad nativo** con latencia ultra baja, cero falsos positivos en los encoders y soporte para selectores de posición (multinivel estilo F1) sin necesidad de modificar el montaje físico.

Especificaciones de Hardware y Pinout Físico

El proyecto mantiene una distribución fija de pines. **No es necesario modificar el cableado** al pasar de una versión de firmware a otra.

- **Microcontrolador:** Raspberry Pi Pico W (RP2040)
- **Pines de Botones y Levas (GP0 a GP11):** 12 entradas digitales con Pull-Up interna (activadas al conectar a `GND`).
    - `GP0` a `GP3`: 4 Levas (Cambios y aux).
    - `GP4` a `GP11`: 8 Botones frontales.
- **Pines de Encoders Rotativos (Pines A y B):**
    - **Encoder 1:** `GP14` (Pin A), `GP15` (Pin B). *(Encoder multinivel / Selector F1 / Normal)*
    - **Encoder 2:** `GP16` (Pin A), `GP17` (Pin B).
    - **Encoder 3:** `GP18` (Pin A), `GP19` (Pin B).
- **Pines de Switches Internos de Encoders (Pulsadores al presionar perilla):**
    - **Switch Encoder 1 (GP20):** Utilizado como modificador `Fn` en la versión final.
    - **Switch Encoder 2 (GP21):** Mapeado como botón digital en Windows.
    - **Switch Encoder 3 (GP22):** Mapeado como botón digital en Windows.

Configuración Previa del Entorno (Arduino IDE)

Para compilar cualquiera de los siguientes códigos, asegúrate de configurar el Arduino IDE de la siguiente manera:

1. **Gestor de Tarjetas:** Instalar el núcleo **Raspberry Pi Pico/RP2040** por Earle F. Philhower.

2. **Selección de Placa:** `Herramientas` > `Placa` > `Raspberry Pi Pico / Pico W`.
3. **Pila USB (CRÍTICO):** `Herramientas` > `USB Stack` > `Adafruit TinyUSB`.
4. **Librerías Requeridas:** Instalar desde el gestor de librerías `Adafruit TinyUSB Library`.

Historial de Versiones del Firmware (Código Completo por Fases)

🔹 Fase 1: Firmware Base Operativo

Características:

- Implementa 12 botones físicos (levas y frontales).
- 3 Encoders rotativos que emiten pulsos rápidos de 30 ms (Botones virtuales 13 al 18).
- Lectura básica de cuadratura por software sin depender de librerías de chips AVR.

```C/C++
#include <Adafruit_TinyUSB.h>

// Descriptor HID personalizado para 32 botones (Report ID 1 + 4 bytes)
uint8_t const desc_hid_report[] = {
  0x05, 0x01,  // Usage Page (Generic Desktop Ctrls)
  0x09, 0x05,  // Usage (Game Pad)
  0xA1, 0x01,  // Collection (Application)
  0x85, 0x01,  //   Report ID (1)
  0x05, 0x09,  //   Usage Page (Button)
  0x19, 0x01,  //   Usage Minimum (Button 1)
  0x29, 0x20,  //   Usage Maximum (Button 32)
  0x15, 0x00,  //   Logical Minimum (0)
  0x25, 0x01,  //   Logical Maximum (1)
  0x75, 0x01,  //   Report Size (1)
  0x95, 0x20,  //   Report Count (32)
  0x81, 0x02,  //   Input (Data, Var, Abs)
  0xC0         // End Collection
};

Adafruit_USBD_HID usb_hid(desc_hid_report, sizeof(desc_hid_report),
HID_ITF_PROTOCOL_NONE, 2, false);

struct WheelReport {
  uint8_t report_id;
  uint8_t buttons[4];
```

```
};

WheelReport gp_report;

class RotaryEncoder {
  uint8_t pinA, pinB;
  uint8_t last_state;
  long count;
public:
  RotaryEncoder(uint8_t a, uint8_t b) {
    pinA = a; pinB = b;
    pinMode(pinA, INPUT_PULLUP);
    pinMode(pinB, INPUT_PULLUP);
    last_state = (digitalRead(pinA) << 1) | digitalRead(pinB);
    count = 0;
  }

  void tick() {
    uint8_t s = (digitalRead(pinA) << 1) | digitalRead(pinB);
    if (s != last_state) {
      if ((last_state == 0 && s == 1) || (last_state == 1 && s == 3) ||
          (last_state == 3 && s == 2) || (last_state == 2 && s == 0)) {
        count++;
      } else if ((last_state == 0 && s == 2) || (last_state == 2 && s == 3) ||
                 (last_state == 3 && s == 1) || (last_state == 1 && s == 0)) {
        count--;
      }
      last_state = s;
    }
  }

  long read() { return count / 4; }
};

const int buttonPins[12] = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11 };

RotaryEncoder enc1(14, 15);
RotaryEncoder enc2(16, 17);
RotaryEncoder enc3(18, 19);

long oldEnc1Pos = 0, oldEnc2Pos = 0, oldEnc3Pos = 0;
unsigned long encTimer1 = 0, encTimer2 = 0, encTimer3 = 0;

void setup() {
  usb_hid.begin();
  for (int i = 0; i < 12; i++) pinMode(buttonPins[i], INPUT_PULLUP);

  gp_report.report_id = 1;
```

```
  memset(gp_report.buttons, 0, sizeof(gp_report.buttons));

  while (!TinyUSBDevice.mounted()) delay(10);
}

void loop() {
  enc1.tick(); enc2.tick(); enc3.tick();

  for (int i = 0; i < 12; i++) {
    int byte_idx = i / 8;
    int bit_idx = i % 8;
    if (digitalRead(buttonPins[i]) == LOW) {
      gp_report.buttons[byte_idx] |= (1 << bit_idx);
    } else {
      gp_report.buttons[byte_idx] &= ~(1 << bit_idx);
    }
  }

  long newEnc1 = enc1.read();
  long newEnc2 = enc2.read();
  long newEnc3 = enc3.read();

  if (newEnc1 != oldEnc1Pos) {
    if (newEnc1 > oldEnc1Pos) gp_report.buttons[1] |= (1 << 4);
    else gp_report.buttons[1] |= (1 << 5);
    encTimer1 = millis(); oldEnc1Pos = newEnc1;
  }
  if (millis() - encTimer1 > 30) gp_report.buttons[1] &= ~((1 << 4) | (1 << 5));

  if (newEnc2 != oldEnc2Pos) {
    if (newEnc2 > oldEnc2Pos) gp_report.buttons[1] |= (1 << 6);
    else gp_report.buttons[1] |= (1 << 7);
    encTimer2 = millis(); oldEnc2Pos = newEnc2;
  }
  if (millis() - encTimer2 > 30) gp_report.buttons[1] &= ~((1 << 6) | (1 << 7));

  if (newEnc3 != oldEnc3Pos) {
    if (newEnc3 > oldEnc3Pos) gp_report.buttons[2] |= (1 << 0);
    else gp_report.buttons[2] |= (1 << 1);
    encTimer3 = millis(); oldEnc3Pos = newEnc3;
  }
  if (millis() - encTimer3 > 30) gp_report.buttons[2] &= ~((1 << 0) | (1 << 1));

  usb_hid.sendReport(1, &gp_report.buttons, sizeof(gp_report.buttons));
  delay(10);
}
```

🔹 Fase 2: Integración de Switches de Encoders y Anti-Falsos Positivos

Novedades:

- Integración de los 3 switches internos de los encoders (`GP20`, `GP21`, `GP22`) asignados a los **Botones 19, 20 y 21**.
- Máquina de estados optimizada para filtrar rebotes mecánicos y eliminar falsos giros inversos.
- Reducción del delay a `5 ms` para mayor tasa de muestreo.

```
C/C++
#include <Adafruit_TinyUSB.h>

uint8_t const desc_hid_report[] = {
  0x05, 0x01, 0x09, 0x05, 0xA1, 0x01, 0x85, 0x01,
  0x05, 0x09, 0x19, 0x01, 0x29, 0x20, 0x15, 0x00,
  0x25, 0x01, 0x75, 0x01, 0x95, 0x20, 0x81, 0x02, 0xC0
};

Adafruit_USBD_HID usb_hid(desc_hid_report, sizeof(desc_hid_report),
HID_ITF_PROTOCOL_NONE, 2, false);

struct WheelReport {
  uint8_t report_id;
  uint8_t buttons[4];
};

WheelReport gp_report;

class RotaryEncoder {
  uint8_t pinA, pinB, last_state;
  long count;
public:
  RotaryEncoder(uint8_t a, uint8_t b) {
    pinA = a; pinB = b;
    pinMode(pinA, INPUT_PULLUP);
    pinMode(pinB, INPUT_PULLUP);
    last_state = (digitalRead(pinA) << 1) | digitalRead(pinB);
    count = 0;
  }

  void tick() {
    uint8_t s = (digitalRead(pinA) << 1) | digitalRead(pinB);
    if (s != last_state) {
      if ((last_state == 0 && s == 1) || (last_state == 1 && s == 3) ||
          (last_state == 3 && s == 2) || (last_state == 2 && s == 0)) count++;
      else if ((last_state == 0 && s == 2) || (last_state == 2 && s == 3) ||
```

```
              (last_state == 3 && s == 1) || (last_state == 1 && s == 0))
count--;
      last_state = s;
    }
  }
  long read() { return count / 4; }
};

const int buttonPins[12] = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11 };
const int switchPins[3] = { 20, 21, 22 };

RotaryEncoder enc1(14, 15);
RotaryEncoder enc2(16, 17);
RotaryEncoder enc3(18, 19);

long oldEnc1Pos = 0, oldEnc2Pos = 0, oldEnc3Pos = 0;
unsigned long encTimer1 = 0, encTimer2 = 0, encTimer3 = 0;

void setup() {
  usb_hid.begin();
  for (int i = 0; i < 12; i++) pinMode(buttonPins[i], INPUT_PULLUP);
  for (int i = 0; i < 3; i++) pinMode(switchPins[i], INPUT_PULLUP);
  gp_report.report_id = 1;
  memset(gp_report.buttons, 0, sizeof(gp_report.buttons));
  while (!TinyUSBDevice.mounted()) delay(10);
}

void loop() {
  enc1.tick(); enc2.tick(); enc3.tick();

  // 1. Botones físicos (1 al 12)
  for (int i = 0; i < 12; i++) {
    int byte_idx = i / 8; int bit_idx = i % 8;
    if (digitalRead(buttonPins[i]) == LOW) gp_report.buttons[byte_idx] |= (1 <<
bit_idx);
    else gp_report.buttons[byte_idx] &= ~(1 << bit_idx);
  }

  // 2. Switches de encoders (Botones 19, 20 y 21)
  for (int i = 0; i < 3; i++) {
    int bit_abs = (19 + i) - 1;
    if (digitalRead(switchPins[i]) == LOW) gp_report.buttons[bit_abs / 8] |= (1 <<
(bit_abs % 8));
    else gp_report.buttons[bit_abs / 8] &= ~(1 << (bit_abs % 8));
  }

  // 3. Encoders (Botones 13 al 18)
  long newEnc1 = enc1.read(), newEnc2 = enc2.read(), newEnc3 = enc3.read();
```

```
  if (newEnc1 != oldEnc1Pos) {
    if (newEnc1 > oldEnc1Pos) gp_report.buttons[1] |= (1 << 4);
    else gp_report.buttons[1] |= (1 << 5);
    encTimer1 = millis(); oldEnc1Pos = newEnc1;
  }
  if (millis() - encTimer1 > 30) gp_report.buttons[1] &= ~((1 << 4) | (1 << 5));

  if (newEnc2 != oldEnc2Pos) {
    if (newEnc2 > oldEnc2Pos) gp_report.buttons[1] |= (1 << 6);
    else gp_report.buttons[1] |= (1 << 7);
    encTimer2 = millis(); oldEnc2Pos = newEnc2;
  }
  if (millis() - encTimer2 > 30) gp_report.buttons[1] &= ~((1 << 6) | (1 << 7));

  if (newEnc3 != oldEnc3Pos) {
    if (newEnc3 > oldEnc3Pos) gp_report.buttons[2] |= (1 << 0);
    else gp_report.buttons[2] |= (1 << 1);
    encTimer3 = millis(); oldEnc3Pos = newEnc3;
  }
  if (millis() - encTimer3 > 30) gp_report.buttons[2] &= ~((1 << 0) | (1 << 1));

  usb_hid.sendReport(1, &gp_report.buttons, sizeof(gp_report.buttons));
  delay(5);
}
```

🔹 Fase 3: Optimización USB Condicional y Selector F1 Cíclico

Novedades:

- **Envío Condicional USB:** Solo transmite paquetes HID si detecta cambios de estado, liberando la saturación del bus USB y permitiendo ejecutar el ciclo a máxima frecuencia (200 MHz sin `delay`).
- **Lógica Cíclica (Wrap-around):** Transición continua estilo selector F1 de 12 posiciones (30° por clic basados en 20 pulsos por vuelta). Al superar la posición final salta a la inicial y viceversa.
- **Emisión de pulso momentáneo (30 ms):** Evita que el botón de posición quede stopeado/retenido permanentemente, facilitando el mapeo limpio en simuladores.

```
C/C++
#include <Adafruit_TinyUSB.h>

uint8_t const desc_hid_report[] = {
```

```
  0x05, 0x01, 0x09, 0x05, 0xA1, 0x01, 0x85, 0x01,
  0x05, 0x09, 0x19, 0x01, 0x29, 0x20, 0x15, 0x00,
  0x25, 0x01, 0x75, 0x01, 0x95, 0x20, 0x81, 0x02, 0xC0
};

Adafruit_USBD_HID usb_hid(desc_hid_report, sizeof(desc_hid_report),
HID_ITF_PROTOCOL_NONE, 2, false);

struct WheelReport { uint8_t report_id; uint8_t buttons[4]; };
WheelReport gp_report;
uint8_t last_sent_buttons[4] = {0, 0, 0, 0};

class RotaryEncoder {
  uint8_t pinA, pinB, last_state;
  long count;
public:
  RotaryEncoder(uint8_t a, uint8_t b) {
    pinA = a; pinB = b;
    pinMode(pinA, INPUT_PULLUP); pinMode(pinB, INPUT_PULLUP);
    last_state = (digitalRead(pinA) << 1) | digitalRead(pinB);
    count = 0;
  }
  void tick() {
    uint8_t s = (digitalRead(pinA) << 1) | digitalRead(pinB);
    if (s != last_state) {
      if ((last_state == 0 && s == 1) || (last_state == 1 && s == 3) ||
          (last_state == 3 && s == 2) || (last_state == 2 && s == 0)) count++;
      else if ((last_state == 0 && s == 2) || (last_state == 2 && s == 3) ||
               (last_state == 3 && s == 1) || (last_state == 1 && s == 0))
count--;
      last_state = s;
    }
  }
  long read() { return count; }
};

const int buttonPins[12] = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11 };
const int switchPins[3] = { 20, 21, 22 };

RotaryEncoder enc1(14, 15);
RotaryEncoder enc2(16, 17);
RotaryEncoder enc3(18, 19);

int f1_position = 0;
long oldDetent1 = 0;
const int max_f1_positions = 12; // 12 estados (30°)
unsigned long f1Timer = 0;
int activeF1Button = -1;
```

```
long oldEnc2Pos = 0, oldEnc3Pos = 0;
unsigned long encTimer2 = 0, encTimer3 = 0;

void setup() {
  usb_hid.begin();
  for (int i = 0; i < 12; i++) pinMode(buttonPins[i], INPUT_PULLUP);
  for (int i = 0; i < 3; i++) pinMode(switchPins[i], INPUT_PULLUP);
  gp_report.report_id = 1;
  memset(gp_report.buttons, 0, sizeof(gp_report.buttons));
  memset(last_sent_buttons, 0, sizeof(last_sent_buttons));
  while (!TinyUSBDevice.mounted()) delay(10);
}

void loop() {
  enc1.tick(); enc2.tick(); enc3.tick();

  // 1. Botones físicos 1 a 12
  for (int i = 0; i < 12; i++) {
    int byte_idx = i / 8, bit_idx = i % 8;
    if (digitalRead(buttonPins[i]) == LOW) gp_report.buttons[byte_idx] |= (1 <<
bit_idx);
    else gp_report.buttons[byte_idx] &= ~(1 << bit_idx);
  }

  // 2. Switches 25, 26, 27
  for (int i = 0; i < 3; i++) {
    int bit_abs = (25 + i) - 1;
    if (digitalRead(switchPins[i]) == LOW) gp_report.buttons[bit_abs / 8] |= (1 <<
(bit_abs % 8));
    else gp_report.buttons[bit_abs / 8] &= ~(1 << (bit_abs % 8));
  }

  // 3. Encoder 1 Cíclico (30° por sección, divisibilidad entre 2)
  long currentDetent1 = enc1.read() / 2;
  if (currentDetent1 != oldDetent1) {
    if (currentDetent1 > oldDetent1) f1_position++;
    else f1_position--;

    if (f1_position < 0) f1_position = max_f1_positions - 1;
    else if (f1_position >= max_f1_positions) f1_position = 0;

    oldDetent1 = currentDetent1;
    activeF1Button = 13 + f1_position; // Botones 13 al 24
    f1Timer = millis();
  }

  // Limpieza global preventiva (evita atascamiento de botones)
```

```
  for (int b = 13; b <= 24; b++) {
    int bit_abs = b - 1;
    gp_report.buttons[bit_abs / 8] &= ~(1 << (bit_abs % 8));
  }

  if (activeF1Button != -1 && (millis() - f1Timer < 30)) {
    int bit_abs_f1 = activeF1Button - 1;
    gp_report.buttons[bit_abs_f1 / 8] |= (1 << (bit_abs_f1 % 8));
  } else if (millis() - f1Timer >= 30) {
    activeF1Button = -1;
  }

  // Encoders 2 y 3
  long newEnc2 = enc2.read() / 4, newEnc3 = enc3.read() / 4;
  if (newEnc2 != oldEnc2Pos) {
    if (newEnc2 > oldEnc2Pos) gp_report.buttons[3] |= (1 << 0);
    else gp_report.buttons[3] |= (1 << 1);
    encTimer2 = millis(); oldEnc2Pos = newEnc2;
  }
  if (millis() - encTimer2 > 30) gp_report.buttons[3] &= ~((1 << 0) | (1 << 1));

  if (newEnc3 != oldEnc3Pos) {
    if (newEnc3 > oldEnc3Pos) gp_report.buttons[3] |= (1 << 2);
    else gp_report.buttons[3] |= (1 << 4);
    encTimer3 = millis(); oldEnc3Pos = newEnc3;
  }
  if (millis() - encTimer3 > 30) gp_report.buttons[3] &= ~((1 << 2) | (1 << 4));

  // Envío condicional USB
  bool hasChanged = false;
  for (int i = 0; i < 4; i++) {
    if (gp_report.buttons[i] != last_sent_buttons[i]) {
      hasChanged = true; break;
    }
  }
  if (hasChanged) {
    usb_hid.sendReport(1, &gp_report.buttons, sizeof(gp_report.buttons));
    memcpy(last_sent_buttons, gp_report.buttons, 4);
  }
}
```

🏆 Fase 4: Firmware Definitivo (Doble Nivel por Software + Modificador Fn)

Características Finales:

1. **Holgura de Lectura (Button Stretch):** Retención mínima de **40 ms** para los 12 botones físicos, asegurando que toques ultra rápidos a las levas se registren limpiamente sin perder pulsaciones.
2. **Transformación del Switch 1 en Botón Fn:** El pin GP20 se excluye de los reportes USB directos para funcionar exclusivamente como tecla modificadora por software.
3. **Nivel 1 (Giro libre):** Al girar el Encoder 1 sin presionar Fn, envía pulsos normales de giro (Botones 13 y 14).
4. **Nivel 2 (Selector de Posición Cíclico de 5 Estados - A, B, C, D, E):**
    - Requiere **mantener presionado Fn y girar al menos 60°**.
    - Emitirá un **único cambio de estado por pulsación del switch Fn**, bloqueando cambios múltiples accidentales en carrera.
    - Ventana del pulso extendida a **60 ms** (ergonomía táctil).

```
C/C++
#include <Adafruit_TinyUSB.h>

// Descriptor HID personalizado para 32 botones (Report ID 1 + 4 bytes)
uint8_t const desc_hid_report[] = {
  0x05, 0x01,  // Usage Page (Generic Desktop Ctrls)
  0x09, 0x05,  // Usage (Game Pad)
  0xA1, 0x01,  // Collection (Application)
  0x85, 0x01,  //   Report ID (1)
  0x05, 0x09,  //   Usage Page (Button)
  0x19, 0x01,  //   Usage Minimum (Button 1)
  0x29, 0x20,  //   Usage Maximum (Button 32)
  0x15, 0x00,  //   Logical Minimum (0)
  0x25, 0x01,  //   Logical Maximum (1)
  0x75, 0x01,  //   Report Size (1)
  0x95, 0x20,  //   Report Count (32)
  0x81, 0x02,  //   Input (Data, Var, Abs)
  0xC0         // End Collection
};

Adafruit_USBD_HID usb_hid(desc_hid_report, sizeof(desc_hid_report),
HID_ITF_PROTOCOL_NONE, 2, false);

struct WheelReport {
  uint8_t report_id;
  uint8_t buttons[4];
};

WheelReport gp_report;
uint8_t last_sent_buttons[4] = {0, 0, 0, 0};

// Clase RotaryEncoder optimizada por cuadratura
class RotaryEncoder {
  uint8_t pinA, pinB;
```

```
  uint8_t last_state;
  long count;
public:
  RotaryEncoder(uint8_t a, uint8_t b) {
    pinA = a; pinB = b;
    pinMode(pinA, INPUT_PULLUP);
    pinMode(pinB, INPUT_PULLUP);
    last_state = (digitalRead(pinA) << 1) | digitalRead(pinB);
    count = 0;
  }

  void tick() {
    uint8_t s = (digitalRead(pinA) << 1) | digitalRead(pinB);
    if (s != last_state) {
      if ((last_state == 0 && s == 1) || (last_state == 1 && s == 3) ||
          (last_state == 3 && s == 2) || (last_state == 2 && s == 0)) {
        count++;
      } else if ((last_state == 0 && s == 2) || (last_state == 2 && s == 3) ||
                 (last_state == 3 && s == 1) || (last_state == 1 && s == 0)) {
        count--;
      }
      last_state = s;
    }
  }

  long read() { return count / 4; }
  void resetCount() { count = 0; }
};

// Configuración de Pines (Hardware intacto)
const int buttonPins[12] = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11 }; // GP0 a GP11
(12 Botones físicos/levas)
const int switchPins[3] = { 20, 21, 22 };                            // GP20 (Fn),
GP21, GP22

RotaryEncoder enc1(14, 15); // Encoder 1 (Doble nivel + Fn)
RotaryEncoder enc2(16, 17); // Encoder 2 (Normal)
RotaryEncoder enc3(18, 19); // Encoder 3 (Normal)

// Temporizadores de holgura para los 12 botones físicos (evita perder pulsos
rápidos)
unsigned long buttonReleaseTimer[12] = {0};

// Variables para Encoder 1 (Doble Nivel con Fn y restricción estricta)
long oldEnc1Pos = 0;
unsigned long enc1TimerNorm = 0;

int f1_state = 0;                 // 5 Estados (0 a 4 -> A, B, C, D, E)
```

```
const int max_f1_states = 5;
unsigned long f1Timer = 0;
int activeF1Button = -1;

// Variables de control estricto para el modo Fn
bool fnWasPressed = false;
bool stateChangedThisPress = false;
long encoderBasePos = 0;

// Variables para Encoders 2 y 3
long oldEnc2Pos = 0;
long oldEnc3Pos = 0;
unsigned long encTimer2 = 0;
unsigned long encTimer3 = 0;

void setup() {
  usb_hid.begin();

  for (int i = 0; i < 12; i++) pinMode(buttonPins[i], INPUT_PULLUP);
  for (int i = 0; i < 3; i++) pinMode(switchPins[i], INPUT_PULLUP);

  gp_report.report_id = 1;
  memset(gp_report.buttons, 0, sizeof(gp_report.buttons));
  memset(last_sent_buttons, 0, sizeof(last_sent_buttons));

  while (!TinyUSBDevice.mounted()) {
    delay(10);
  }
}

void loop() {
  // 1. Muestreo continuo sin retardo
  enc1.tick();
  enc2.tick();
  enc3.tick();

  // 2. Lectura de los 12 Botones Físicos con sistema de holgura (40ms mínimo
activos)
  for (int i = 0; i < 12; i++) {
    int byte_idx = i / 8;
    int bit_idx = i % 8;
    if (digitalRead(buttonPins[i]) == LOW) {
      gp_report.buttons[byte_idx] |= (1 << bit_idx);
      buttonReleaseTimer[i] = millis() + 40; // Asegura visibilidad de toques
rápidos
    } else {
      if (millis() > buttonReleaseTimer[i]) {
        gp_report.buttons[byte_idx] &= ~(1 << bit_idx);
```

```
      }
    }
  }

  // 3. Lectura de los Switches 2 y 3 (GP21 y GP22) -> El GP20 (switch 1) ahora es
Fn y no se reporta
  for (int i = 1; i < 3; i++) { // Empieza en 1 para omitir GP20
    int btn_num = 23 + i; // Botones 24 y 25
    int bit_abs = btn_num - 1;
    if (digitalRead(switchPins[i]) == LOW) {
      gp_report.buttons[bit_abs / 8] |= (1 << (bit_abs % 8));
    } else {
      gp_report.buttons[bit_abs / 8] &= ~(1 << (bit_abs % 8));
    }
  }

  // --- 4. ENCODER 1: LÓGICA DE DOBLE NIVEL CON CONTROL FN Y 60° ---
  long newEnc1 = enc1.read();
  bool isFnPressed = (digitalRead(switchPins[0]) == LOW); // GP20 actúa como
modificador Fn

  if (isFnPressed) {
    // --- GESTIÓN DEL MODO SELECTOR (NIVEL 2) ---
    if (!fnWasPressed) {
      // Acaba de ser presionado el botón Fn: capturamos posición base y
habilitamos 1 solo cambio
      fnWasPressed = true;
      stateChangedThisPress = false;
      encoderBasePos = newEnc1;
    }

    // Si aún no se ha realizado el cambio de estado para esta pulsación de Fn
    if (!stateChangedThisPress) {
      long diffFromBase = newEnc1 - encoderBasePos;

      // Umbral de 60° (aprox 3 cuentas de encoder)
      if (abs(diffFromBase) >= 3) {
        if (diffFromBase > 0) {
          f1_state++;
        } else {
          f1_state--;
        }

        // Ciclo continuo (Wrap-around: A -> B -> C -> D -> E -> A...)
        if (f1_state < 0) f1_state = max_f1_states - 1;
        else if (f1_state >= max_f1_states) f1_state = 0;
```

```
        activeF1Button = 15 + f1_state; // Mapeado a Botones 15 a 19 (Estados A al
E)
        f1Timer = millis();

        // BLOQUEO ESTRICTO: Marcamos que ya cambió 1 estado; no permitirá más
hasta soltar Fn
        stateChangedThisPress = true;
      }
    }
    oldEnc1Pos = newEnc1; // Sincroniza posición
  }
  else {
    // --- GESTIÓN DE USO NORMAL (NIVEL 1) CUANDO NO SE PRESIONA FN ---
    fnWasPressed = false;          // Resetea bandera al soltar Fn
    stateChangedThisPress = false;

    if (newEnc1 != oldEnc1Pos) {
      long diff = newEnc1 - oldEnc1Pos;
      if (diff > 0) {
        gp_report.buttons[1] |= (1 << 4); // Botón 13
      } else {
        gp_report.buttons[1] |= (1 << 5); // Botón 14
      }
      enc1TimerNorm = millis();
      oldEnc1Pos = newEnc1;
    }
  }

  // Apagado del Nivel 1 a los 60ms
  if (millis() - enc1TimerNorm > 60) {
    gp_report.buttons[1] &= ~((1 << 4) | (1 << 5));
  }

  // Apagado y limpieza estricta del Nivel 2 (Estados A al E) con pulso de 60ms
  for (int b = 15; b <= 19; b++) {
    int bit_abs = b - 1;
    gp_report.buttons[bit_abs / 8] &= ~(1 << (bit_abs % 8));
  }
  if (activeF1Button != -1 && (millis() - f1Timer < 60)) {
    int bit_abs_f1 = activeF1Button - 1;
    gp_report.buttons[bit_abs_f1 / 8] |= (1 << (bit_abs_f1 % 8));
  } else if (millis() - f1Timer >= 60) {
    activeF1Button = -1;
  }

  // --- 5. ENCODERS 2 Y 3 (PULSOS NORMALES CON PULSO DE 60ms) ---
  long newEnc2 = enc2.read();
  long newEnc3 = enc3.read();
```

```
  // Encoder 2 -> Botones 20 y 21
  if (newEnc2 != oldEnc2Pos) {
    if (newEnc2 > oldEnc2Pos) gp_report.buttons[2] |= (1 << 3);  // Botón 20
    else gp_report.buttons[2] |= (1 << 4);                       // Botón 21
    encTimer2 = millis();
    oldEnc2Pos = newEnc2;
  }
  if (millis() - encTimer2 > 60) {
    gp_report.buttons[2] &= ~((1 << 3) | (1 << 4));
  }

  // Encoder 3 -> Botones 22 y 23
  if (newEnc3 != oldEnc3Pos) {
    if (newEnc3 > oldEnc3Pos) gp_report.buttons[2] |= (1 << 5);  // Botón 22
    else gp_report.buttons[2] |= (1 << 6);                       // Botón 23
    encTimer3 = millis();
    oldEnc3Pos = newEnc3;
  }
  if (millis() - encTimer3 > 60) {
    gp_report.buttons[2] &= ~((1 << 5) | (1 << 6));
  }

  // --- 6. ENVÍO CONDICIONAL USB (OPTIMIZACIÓN MÁXIMA DE LATENCIA) ---
  bool hasChanged = false;
  for (int i = 0; i < 4; i++) {
    if (gp_report.buttons[i] != last_sent_buttons[i]) {
      hasChanged = true;
      break;
    }
  }

  if (hasChanged) {
    usb_hid.sendReport(1, &gp_report.buttons, sizeof(gp_report.buttons));
    memcpy(last_sent_buttons, gp_report.buttons, 4);
  }
}
```

Mapeo de Botones en Windows (`joy.cpl`) - Firmware Final

Acción Física	**Botón Asignado en Windows**	**Notas / Tipo de Señal**
Levas y Botones Físicos (GP0 - GP11)	**Botones 1 al 12**	Entrada retenida (mientras se mantenga presionado).

Acción Física	Botón Asignado en Windows	Notas / Tipo de Señal
Encoder 1 - Giro Normal (Izq / Der)	**Botones 13 y 14**	Pulso temporal de 60 ms.
Encoder 1 - Selector Nivel 2 (Estados A-E)	**Botones 15 a 19**	Pulso único de 60 ms (Requiere `Fn` + Giro 60°).
Encoder 2 - Giro Normal (Izq / Der)	**Botones 20 y 21**	Pulso temporal de 60 ms.
Encoder 3 - Giro Normal (Izq / Der)	**Botones 22 y 23**	Pulso temporal de 60 ms.
Switch Encoder 1 (GP20)	*Sin asignar (Modificador Fn)*	Se usa internamente para habilitar Nivel 2.
Switch Encoder 2 (GP21)	**Botón 24**	Entrada retenida digital.
Switch Encoder 3 (GP22)	**Botón 25**	Entrada retenida digital.

Guía de Pruebas y Validación

1. Conecta la Raspberry Pi Pico W por puerto USB a la computadora.
2. Abre la ventana de dispositivos de juego de Windows presionando `Win + R`, escribiendo `joy.cpl` y presionando `Enter`.
3. Selecciona el gamepad reconocido y abre sus **Propiedades**.
4. Presiona rápidamente cualquier leva: comprobarás que se ilumina inmediatamente sin perder pulsaciones.
5. Mantén presionado el switch del **Encoder 1 (GP20)**: observarás que **ningún botón se enciende en Windows**.
6. Con el switch del Encoder 1 presionado, gira la perilla 60°: se encenderá el botón correspondiente al siguiente estado durante 60 ms y se apagará. Si continúas girando sin soltar el switch, no cambiará más veces, garantizando precisión total.